

COMPTE RENDU TECHNIQUE

Implémentation d'un tunnel VPN WireGuard

Pour l'administration sécurisée d'une infrastructure ESP32

Karadag Nissa

Projet	wireguard-vpn-esp32-admin
Environnement serveur	Ubuntu 24.04 LTS — VM VirtualBox
Environnement client	Windows 11 — WireGuard GUI
Date	Décembre 2025

1. Contexte et objectifs

Ce projet a pour objectif de mettre en place un accès administrateur sécurisé à une machine virtuelle Ubuntu (serveur ESP32) depuis un poste client Windows, sans exposer directement les services sensibles (SSH, MQTT) sur le réseau local.

Pour cela, un tunnel VPN WireGuard a été déployé afin de créer un réseau privé virtuel chiffré (10.10.10.0/24), dans lequel seuls les clients authentifiés par paire de clés cryptographiques peuvent accéder aux services d'administration.

1.1 Contraintes

- Ne pas exposer le port SSH (22) ni MQTT (8883) directement sur le réseau local
- Maintenir un accès distant fiable et chiffré de bout en bout
- Utiliser un protocole VPN moderne, léger et performant
- Supprimer la clé privée client du serveur après configuration

2. Choix technique : WireGuard

WireGuard a été retenu comme solution VPN pour les raisons suivantes :

Critère	Justification
Légèreté	~4 000 lignes de code vs des dizaines de milliers pour OpenVPN ou IPsec
Performance	Protocole UDP avec cryptographie moderne : Curve25519, ChaCha20, Poly1305
Simplicité	Un seul fichier .conf par pair, pas de PKI complexe à gérer
Sécurité	Authentification par clés asymétriques, pas de négociation exposée
Intégration Linux	Intégré au noyau Linux depuis la version 5.6, performances optimales

3. Architecture réseau

Le tunnel VPN établit un réseau privé virtuel entre le poste client et la VM serveur. Toutes les communications d'administration transitent par ce tunnel chiffré.

Composant	Configuration réseau
VM Ubuntu (Serveur)	enp0s3 : 192.168.1.183/24 wg0 : 10.10.10.1/24
PC Windows (Client)	Interface physique : 192.168.1.x wg-vm : 10.10.10.2/32
Réseau VPN	10.10.10.0/24 — chiffré de bout en bout

Port d'écoute WireGuard	UDP 51820
-------------------------	-----------

Le trafic SSH transite uniquement via l'adresse VPN 10.10.10.1.

Toute tentative de connexion SSH directe sur 192.168.1.183 est rejetée par le pare-feu.

4. Déroulement de l'implémentation

4.1 Installation de WireGuard

WireGuard a été installé sur la VM Ubuntu via le gestionnaire de paquets APT :

```
sudo apt update
sudo apt install wireguard -y
```

```
esp32@esp32-VirtualBox:~$ sudo apt install wireguard -y
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Le paquet suivant a été installé automatiquement et n'est plus nécessaire :
  libllvm19
Veuillez utiliser « sudo apt autoremove » pour le supprimer.
```

Figure 1 — Installation de WireGuard via APT

4.2 Génération et sécurisation des clés serveur

Avant la génération des clés, le umask a été positionné à 077 afin que les nouveaux fichiers héritent automatiquement des permissions 600 (lecture/écriture propriétaire uniquement). La commande `wg genkey` génère la clé privée, dont la clé publique est dérivée par `wg pubkey`.

```
umask 077
wg genkey | tee server.key | wg pubkey > server.pub
```

```
esp32@esp32-VirtualBox:~$ umask 077
esp32@esp32-VirtualBox:~$ wg genkey | tee server.key | wg pubkey > server.pub
esp32@esp32-VirtualBox:~$ |
```

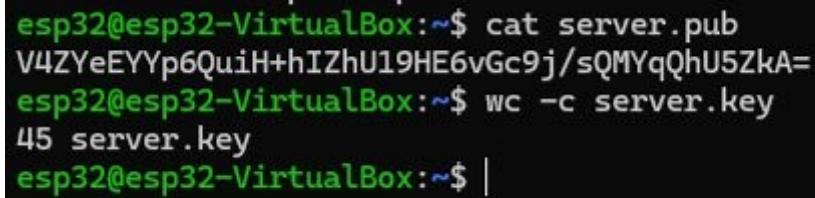
Figure 2 — Génération de la paire de clés serveur

La vérification des permissions et de l'existence des fichiers confirme que les droits sont correctement appliqués :

```
esp32@esp32-VirtualBox:~$ ls -l server.key server.pub
-rw----- 1 esp32 esp32 45 déc. 20 12:06 server.key
-rw----- 1 esp32 esp32 45 déc. 20 12:06 server.pub
esp32@esp32-VirtualBox:~$ |
```

Figure 3 — Vérification des permissions (rw-----)

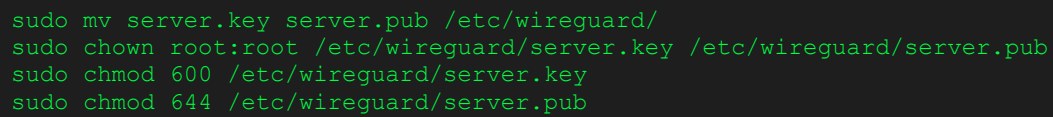
La clé publique est affichée pour être partagée avec les clients. La longueur de la clé privée (45 caractères) est contrôlée sans jamais en afficher le contenu :



```
esp32@esp32-VirtualBox:~$ cat server.pub
V4ZYeEYYp6QuiH+hIZhU19HE6vGc9j/sQMYqQhU5ZkA=
esp32@esp32-VirtualBox:~$ wc -c server.key
45 server.key
esp32@esp32-VirtualBox:~$ |
```

Figure 4 — Affichage de la clé publique et contrôle de longueur

Les fichiers ont ensuite été déplacés dans /etc/wireguard/ avec les droits appropriés :



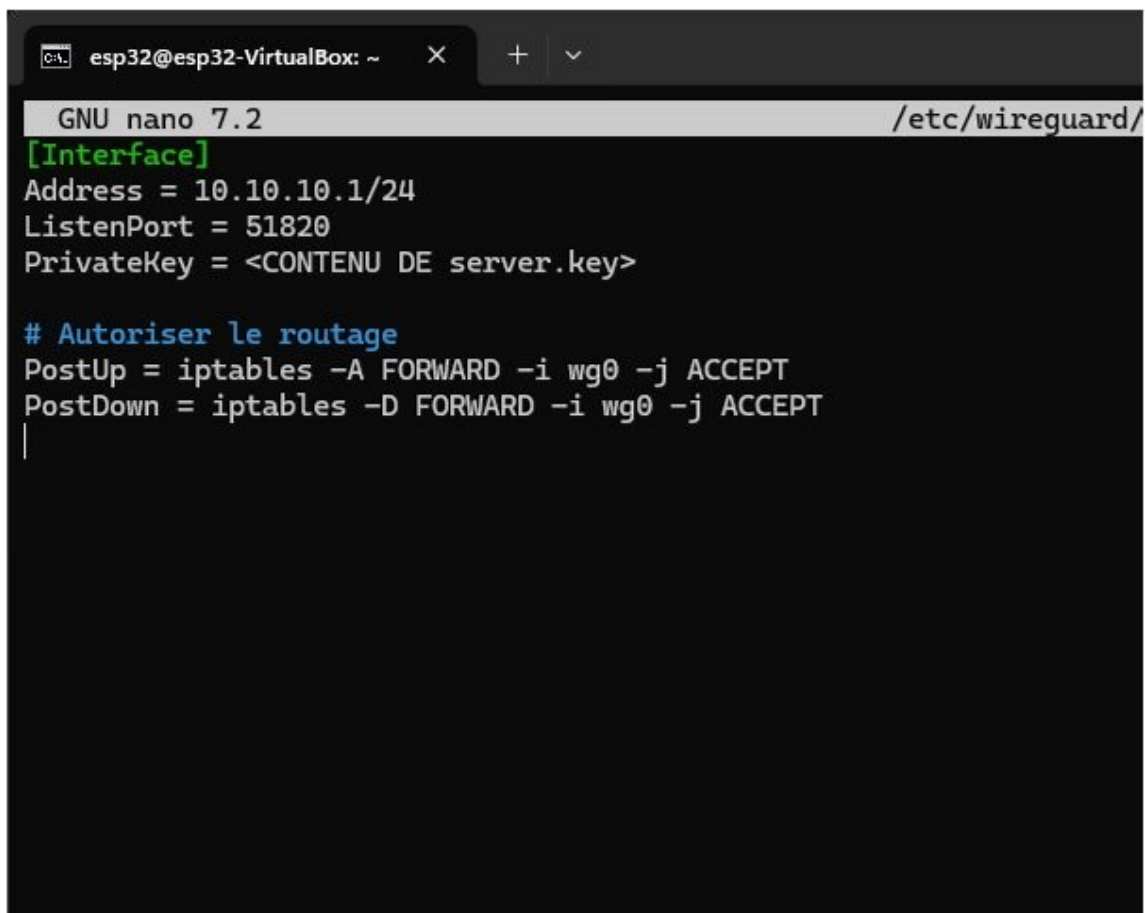
```
sudo mv server.key server.pub /etc/wireguard/
sudo chown root:root /etc/wireguard/server.key /etc/wireguard/server.pub
sudo chmod 600 /etc/wireguard/server.key
sudo chmod 644 /etc/wireguard/server.pub
```

4.3 Configuration du serveur WireGuard

Le fichier `/etc/wireguard/wg0.conf` a été créé avec les paramètres de l'interface VPN côté serveur. Les règles `PostUp` et `PostDown` permettent le routage du trafic entrant sur l'interface `wg0` via `iptables`.

```
[Interface]
Address = 10.10.10.1/24
ListenPort = 51820
PrivateKey = <CONTENU DE server.key>

# Autoriser le routage
PostUp = iptables -A FORWARD -i wg0 -j ACCEPT
PostDown = iptables -D FORWARD -i wg0 -j ACCEPT
```



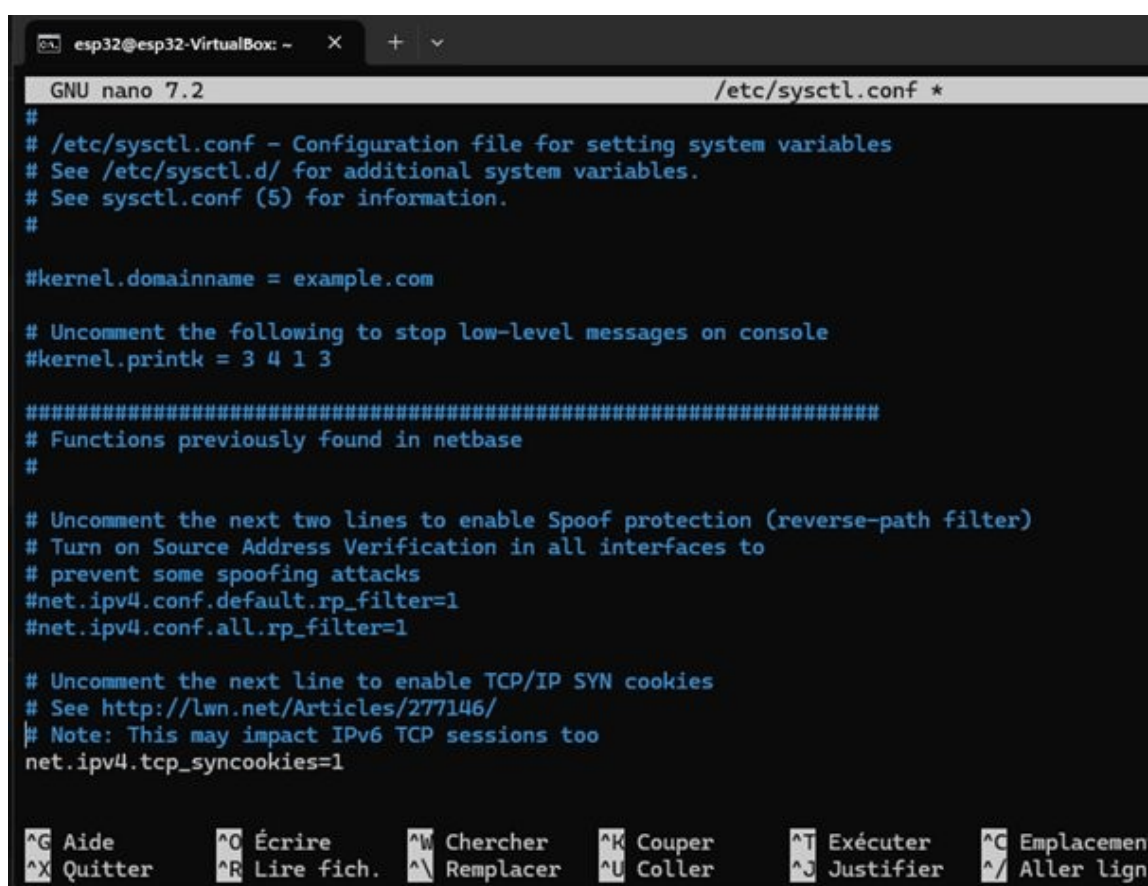
```
esp32@esp32-VirtualBox: ~
GNU nano 7.2 /etc/wireguard/
[Interface]
Address = 10.10.10.1/24
ListenPort = 51820
PrivateKey = <CONTENU DE server.key>

# Autoriser le routage
PostUp = iptables -A FORWARD -i wg0 -j ACCEPT
PostDown = iptables -D FORWARD -i wg0 -j ACCEPT
|
```

Figure 5 — Contenu du fichier `wg0.conf` dans `nano`

4.4 Activation du routage IP (IP Forwarding)

Pour permettre la circulation du trafic VPN entre les interfaces réseau, le routage IPv4 a été activé dans `/etc/sysctl.conf` en décommentant la directive `net.ipv4.ip_forward=1`. La commande `sudo sysctl -p` applique le changement sans redémarrage.



```
esp32@esp32-VirtualBox: ~  
GNU nano 7.2 /etc/sysctl.conf *  
#  
# /etc/sysctl.conf - Configuration file for setting system variables  
# See /etc/sysctl.d/ for additional system variables.  
# See sysctl.conf (5) for information.  
#  
#kernel.domainname = example.com  
  
# Uncomment the following to stop low-level messages on console  
#kernel.printk = 3 4 1 3  
  
#####  
# Functions previously found in netbase  
#  
  
# Uncomment the next two lines to enable Spoof protection (reverse-path filter)  
# Turn on Source Address Verification in all interfaces to  
# prevent some spoofing attacks  
#net.ipv4.conf.default.rp_filter=1  
#net.ipv4.conf.all.rp_filter=1  
  
# Uncomment the next line to enable TCP/IP SYN cookies  
# See http://lwn.net/Articles/277146/  
# Note: This may impact IPv6 TCP sessions too  
net.ipv4.tcp_syncookies=1  
  
^G Aide      ^O Écrire    ^W Chercher  ^K Couper    ^T Exécuter  ^C Emplacemen  
^X Quitter   ^R Lire fich.^_ Remplacer  ^U Coller    ^J Justifier  ^_ Aller lign
```

Figure 6 — Activation du forwarding IP dans sysctl.conf

4.5 Ouverture du port firewall (UFW)

Le port UDP 51820, utilisé par WireGuard pour la communication chiffrée, a été autorisé dans le pare-feu UFW :

```
sudo ufw allow 51820/udp
```

```
esp32@esp32-VirtualBox:~$ sudo ufw allow 51820/udp
La règle a été ajoutée
La règle a été ajoutée (v6)
esp32@esp32-VirtualBox:~$ |
```

Figure 7 — Autorisation du port 51820/UDP dans UFW

4.6 Démarrage et activation du service WireGuard

Le service a été activé pour démarrer automatiquement au boot, puis lancé immédiatement :

```
sudo systemctl enable wg-quick@wg0
sudo systemctl start wg-quick@wg0
sudo wg # Vérification de l'interface active
```

```
esp32@esp32-VirtualBox:~$ sudo systemctl enable wg-quick@wg0
Created symlink /etc/systemd/system/multi-user.target.wants/wg-quick@wg0.service → /usr/lib/systemd/system/wg-quick@.service.
esp32@esp32-VirtualBox:~$ sudo systemctl start wg-quick@wg0
esp32@esp32-VirtualBox:~$ |
```

Figure 8 — Activation et démarrage du service wg-quick@wg0

4.7 Création et enregistrement du client PC

Une paire de clés a été générée pour le poste client Windows. La clé publique du client a été ajoutée dans wg0.conf en tant que [Peer], permettant au serveur d'authentifier la connexion.

```
wg genkey | tee pc.key | wg pubkey > pc.pub
```

Ajout du bloc [Peer] dans /etc/wireguard/wg0.conf :

```
[Peer]
PublicKey = <CONTENU DE pc.pub>
AllowedIPs = 10.10.10.2/32
```



```
esp32@esp32-VirtualBox:~$ wg genkey | tee pc.key | wg pubkey > pc.pub
esp32@esp32-VirtualBox:~$ sudo nano /etc/wireguard/wg0.conf
esp32@esp32-VirtualBox:~$ |
```

```
# Autoriser le routage
PostUp = iptables -A FORWARD -i wg0 -j ACCEPT
PostDown = iptables -D FORWARD -i wg0 -j ACCEPT

[Peer]
PublicKey = unBoyNp0yVXdINnR1AW5lOKt+03l/t8V00vZgW/KoXw=
AllowedIPs = 10.10.10.2/32
```

```
esp32@esp32-VirtualBox:~$ sudo nano /etc/wireguard/wg0.conf
[sudo] Mot de passe de esp32 :
esp32@esp32-VirtualBox:~$ sudo systemctl restart wg-quick@wg0
esp32@esp32-VirtualBox:~$ |
```

Figure 9 — Génération des clés client et ajout du Peer dans wg0.conf

Bonne pratique de sécurité : la clé privée du client (pc.key) a été supprimée du serveur après transfert sur le poste Windows. Elle ne doit résider que sur la machine cliente.

4.8 Configuration du tunnel côté Windows

WireGuard a été installé sur le poste Windows (<https://www.wireguard.com/install/>). Un nouveau tunnel nommé wg-vm a été créé avec la configuration suivante. Le paramètre PersistentKeepalive = 25 maintient la connexion active même derrière un NAT.

```
[Interface]
PrivateKey = <CONTENU DE pc.key>
Address = 10.10.10.2/32
DNS = 1.1.1.1

[Peer]
PublicKey = <CONTENU DE server.pub>
Endpoint = 192.168.1.183:51820
AllowedIPs = 10.10.10.0/24
PersistentKeepalive = 25
```

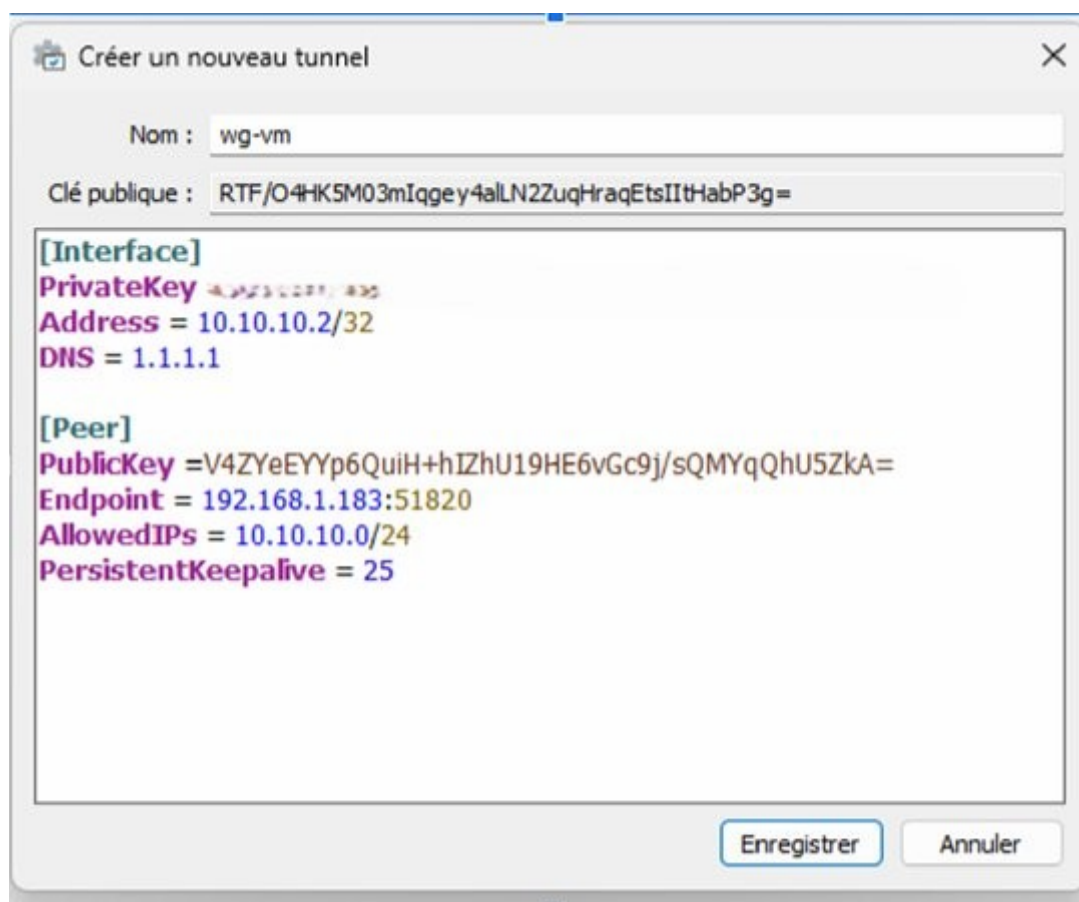


Figure 10 — Interface de création du tunnel WireGuard sous Windows

4.9 Vérification du tunnel actif

La commande `sudo wg` sur le serveur confirme que le pair Windows est bien enregistré et que l'interface wg0 est active sur le port 51820 :

```

esp32@esp32-VirtualBox:~$ sudo wg
interface: wg0
  public key: V4ZYeEYyp6QuiH+hIZhU19HE6vGc9j/sQMYqQhU5ZkA=
  private key: (hidden)
  listening port: 51820

peer: RTF/04HK5M03mIqgey4aLLN2ZuqHraqEtsIIItHabP3g=
  allowed ips: 10.10.10.2/32
esp32@esp32-VirtualBox:~$ |

```

Figure 11 — État de l'interface WireGuard et du pair enregistré

4.10 Restriction SSH au tunnel VPN uniquement

Pour finaliser la sécurisation, les règles UFW ont été modifiées afin d'autoriser SSH exclusivement depuis le sous-réseau VPN et de le bloquer depuis tout autre accès :

```

sudo ufw deny 22
sudo ufw allow from 10.10.10.0/24 to any port 22
sudo ufw deny in to any port 22 proto tcp
sudo ufw reload

```

```

esp32@esp32-VirtualBox:~$ sudo ufw deny 22
La règle a été ajoutée
La règle a été ajoutée (v6)
esp32@esp32-VirtualBox:~$ sudo ufw allow from 10.10.10.0/24 to any port 22
La règle a été ajoutée
esp32@esp32-VirtualBox:~$ sudo ufw reload
Pare-feu rechargé
esp32@esp32-VirtualBox:~$ |

```

Figure 12 — Application des règles SSH restreintes au VPN

```

esp32@esp32-VirtualBox:~$ sudo ufw deny in to any port 22 proto tcp
La règle a été ajoutée
La règle a été ajoutée (v6)
esp32@esp32-VirtualBox:~$ sudo ufw reload
Pare-feu rechargé
esp32@esp32-VirtualBox:~$ sudo ufw status
État : actif

```

Vers	Action	De
8883/tcp	ALLOW	Anywhere
51820/udp	ALLOW	Anywhere
22	ALLOW	10.10.10.0/24
22/tcp	DENY	Anywhere
8883/tcp (v6)	ALLOW	Anywhere (v6)
51820/udp (v6)	ALLOW	Anywhere (v6)
22/tcp (v6)	DENY	Anywhere (v6)

```

esp32@esp32-VirtualBox:~$ |

```

Figure 13 — Tableau récapitulatif des règles UFW actives

5. Tests et validation

5.1 Test sans VPN — comportement attendu

Avec le tunnel VPN désactivé, une tentative de connexion SSH directe sur l'IP physique du serveur (192.168.1.183) résulte en un timeout, confirmant que SSH n'est plus accessible hors tunnel :

```
ssh: connect to host 192.168.1.183 port 22: Connection timed out
```

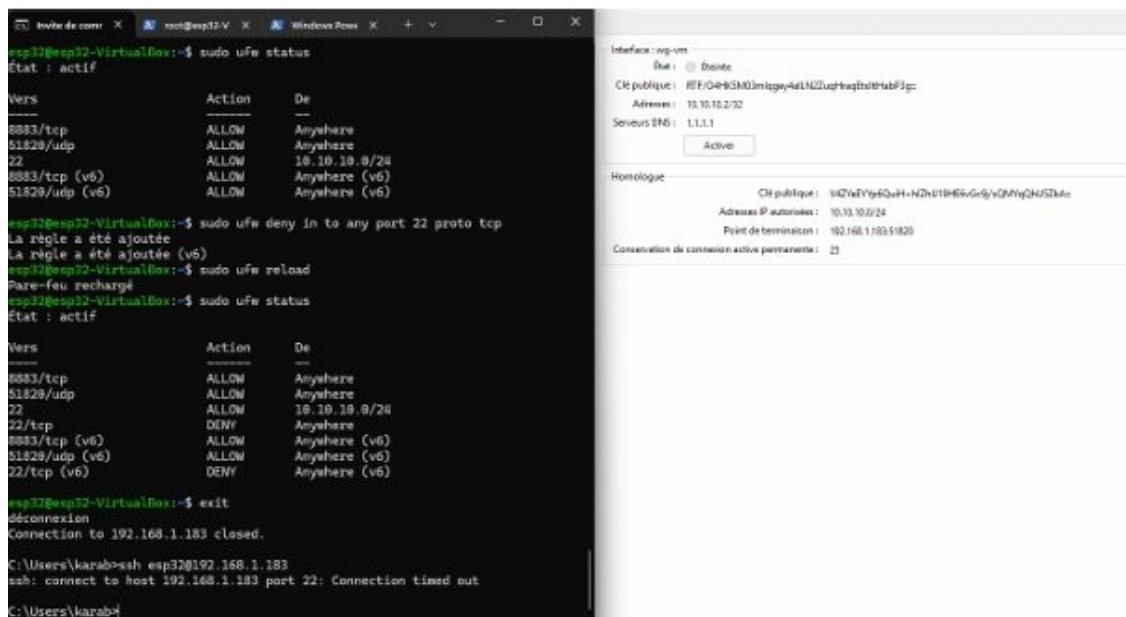


Figure 14 — Connexion SSH refusée sans VPN actif (timeout)

5.2 Test avec VPN actif — succès

Avec le tunnel VPN activé côté Windows, la connexion SSH via l'IP VPN (10.10.10.1) s'établit correctement, accédant au shell Ubuntu du serveur :

```
ssh esp32@10.10.10.1
# Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-37-generic x86_64)
```

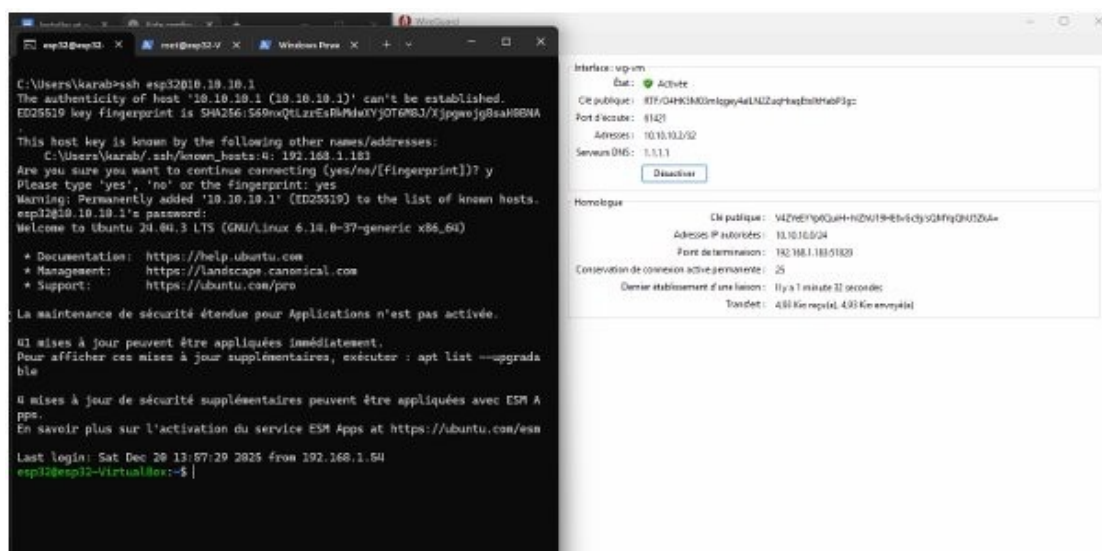


Figure 15 — Connexion SSH réussie via le tunnel VPN (10.10.10.1)

5.3 Vérification des interfaces réseau

La commande `ip a` sur le serveur confirme la présence de l'interface `wg0` avec l'adresse `10.10.10.1/24`, en mode point-à-point :

```
esp32@esp32-VirtualBox:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:65:e7:e7 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.183/24 brd 192.168.1.255 scope global dynamic noprefixroute enp0s3
        valid_lft 34474sec preferred_lft 34474sec
    inet6 2a01:e0a:806:66f0:51aa:9b2d:4f0a:94d7/64 scope global temporary dynamic enp0s3
        valid_lft 86210sec preferred_lft 77442sec
    inet6 2a01:e0a:806:66f0:a00:27ff:fe65:e7e7/64 scope global dynamic mngtmpaddr enp0s3
        valid_lft 86210sec preferred_lft 86210sec
    inet6 fe80::a00:27ff:fe65:e7e7/64 scope link
        valid_lft forever preferred_lft forever
6: wg0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1420 qdisc noqueue state UNKNOWN group default qlen 1000
    link/none
    inet 10.10.10.1/24 scope global wg0
        valid_lft forever preferred_lft forever
esp32@esp32-VirtualBox:~$
```

Figure 16 — Présence de l'interface `wg0` dans la liste des interfaces réseau

6. Bilan et règles appliquées

6.1 Règles UFW finales

Destination	Protocole	Action	Source
8883	TCP	ALLOW	Anywhere
51820	UDP	ALLOW	Anywhere
22	—	ALLOW	10.10.10.0/24 (VPN uniquement)
22	TCP	DENY	Anywhere

6.2 Ce qui a été accompli

- ✓ Tunnel WireGuard fonctionnel entre le client Windows et la VM Ubuntu
- ✓ Réseau privé virtuel 10.10.10.0/24 chiffré et authentifié par clés cryptographiques
- ✓ SSH accessible uniquement via le VPN, inaccessible depuis le réseau physique
- ✓ Surface d'attaque réseau considérablement réduite
- ✓ Service WireGuard activé au démarrage (systemctl enable)

6.3 Limites identifiées

Le VPN constitue une couche de sécurité réseau efficace, mais il ne remplace pas une stratégie de défense en profondeur. Les points suivants ont été identifiés comme limites de la configuration actuelle :

- Le compte esp32 dispose de privilèges sudo : si les identifiants sont compromis, les protections réseau peuvent être contournées par un attaquant.
- L'authentification SSH se fait encore par mot de passe (moins sûr que par clé).
- Aucun système de journalisation des connexions n'est en place (fail2ban, auditd).

6.4 Pistes d'amélioration

- Mettre en place l'authentification SSH par clé et désactiver les mots de passe
- Installer fail2ban pour la détection d'intrusions
- Ajouter auditd pour la journalisation des accès administrateurs
- Automatiser la révocation de pairs clients (script de gestion)
- Envisager un serveur WireGuard dédié, séparé du serveur MQTT/ESP32

7. Références

Ressource	Lien / Source
Documentation officielle WireGuard	https://www.wireguard.com/

WireGuard Whitepaper	https://www.wireguard.com/papers/wireguard.pdf
UFW — Ubuntu documentation	https://help.ubuntu.com/community/UFW
sysctl — Linux man page	https://man7.org/linux/man-pages/man8/sysctl.8.html
Installation Windows	https://www.wireguard.com/install/